



// session 06 beginner Python

One job. One function.

Turn a crowded game loop into clear, named steps.

def

parameters

return

// make the loop readable



Make main() read like a mission plan.

We will extract three focused jobs without changing how the game plays.

01

Scan

Show danger before a choice.

02

Check

Return a defeat cause.

03

Process

Handle one destination.

OUTCOME

You can define, call, and trust your own functions.



```
// retrieve dictionaries
```

What value does this lookup return?

```
destination["danger_level"]
```

when

```
danger_level = 3
```

3

The key selects one value from the destination dictionary.

The loop works— but it is doing too many jobs.

Travel, display, encounters, status, and defeat checks are mixed together.

```
oxygen = oxygen - 8
print(destination["description"])

if should_stop:
    oxygen, hull = process_destination(destination, oxygen,
hull)

print(f"Oxygen: {oxygen} | Hull: {hull}")

if oxygen ≤ 0:
    print("Oxygen depleted")
if hull ≤ 0:
    print("Hull destroyed")
```

A function gives one job a name.

Then the loop can ask for that job without carrying every detail.

show_scan()

describe danger

process_destination()

apply the encounter

check_defeat()

decide whether the game ends

def creates a named block of instructions.

scan function name
destination parameter
indented lines function body

```
def scan(destination):  
    danger = destination["danger_level"]  
    print(f"SCANNER: {destination['name']} –  
    threat level {danger}")
```

```
// call make it run
```

Defining is not calling.

The call passes one dictionary into the parameter named destination.

```
SAFE_PLANET → destination → print
```

```
scan(SAFE_PLANET)
```


STEP 01

One function makes one decision.

The same fixed danger value always prints the same rating.

```
def scan(destination):
    danger = destination["danger_level"]
    if danger > 3:
        print(f"SCANNER: {destination['name']} - DANGEROUS")
    elif danger > 1:
        print(f"SCANNER: {destination['name']} - RISKY")
    else:
        print(f"SCANNER: {destination['name']} - SAFE")
```




// predict three fixed cases

What rating comes back for danger levels 1, 3, and 5?

1

SAFE

3

RISKY

5

DANGEROUS

```
// return send an answer back
```

parameters in
oxygen, hull



function
check_defeat



value back
cause or None

RETURN Hand one answer back to the line that called the function.

STEP 02

Calculate first. Display later.

This function does not print or change resources.

```
def check_defeat(oxygen, hull):  
    if oxygen ≤ 0:  
        return "Oxygen depleted"  
    if hull ≤ 0:  
        return "Hull destroyed"  
    return None
```



// predict trace the return

What comes back from each call?

`check_defeat(0, 100)`

"Oxygen depleted"

`check_defeat(100, 0)`

"Hull destroyed"

`check_defeat(40, 60)`

None

```
// caller decide what to do
```

The function checks. The caller responds.

If cause contains text, the game can show defeat and stop.

```
cause = check_defeat(0, 70)

if cause:
    print(cause)
```

Same inputs. Same answer. No side effects.

A pure function calculates and returns. It does not print or secretly change other data.

```
def oxygen_after_jump(oxygen, jump_cost):  
    return oxygen - jump_cost
```

```
remaining = oxygen_after_jump(100, 8)  
print(remaining)
```

Keep each function's job easy to explain.

// calculates

check_defeat()

Returns a cause or None.

// does

show_scan()

Prints information for the player.

TEST Can you describe the function without using "and"?

STEP 03

Group the encounter and water work.

Updated oxygen and hull return together.

```
def process_destination(destination, oxygen, hull):
    oxygen, hull, narration = process_encounter(
        destination, oxygen, hull
    )
    show_encounter(narration)

    if destination["has_water"]:
        oxygen, hull, water_narration = process_water_planet(
            oxygen, hull
        )
        show_encounter(water_narration)

    return oxygen, hull
```


The main path now tells the story.

Scan. Process. Check. Details live behind useful names.

```
destination = RISKY_PLANET
oxygen = 72
hull = 100

scan(destination)
oxygen, hull = process_destination(destination,
oxygen, hull)
cause = check_defeat(oxygen, hull)
```

scan → encounter → defeat check

Refactor without changing the game.

Use dictionaries, loops, and conditionals you already know.

30 minutes

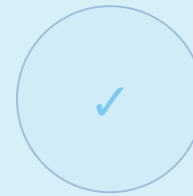
01 Add and call scan(destination).

02 Add and call check_defeat(oxygen, hull).

03 Extract process_destination.

04 Run uv run game.py.

SUCCESS Scanner output appears and every win/loss path still works.



// verify behaviour stayed fixed

Test the branches, not just the happy path.

SCAN

Threat 1, 3, and 5 show the right rating.

RESOURCES

Water and encounters update values.

ENDING

Oxygen, hull, and victory paths work.

Two common function mistakes

MISSING ANSWER

```
return oxygen, hull
```

Check: Does the function hand updated values back?

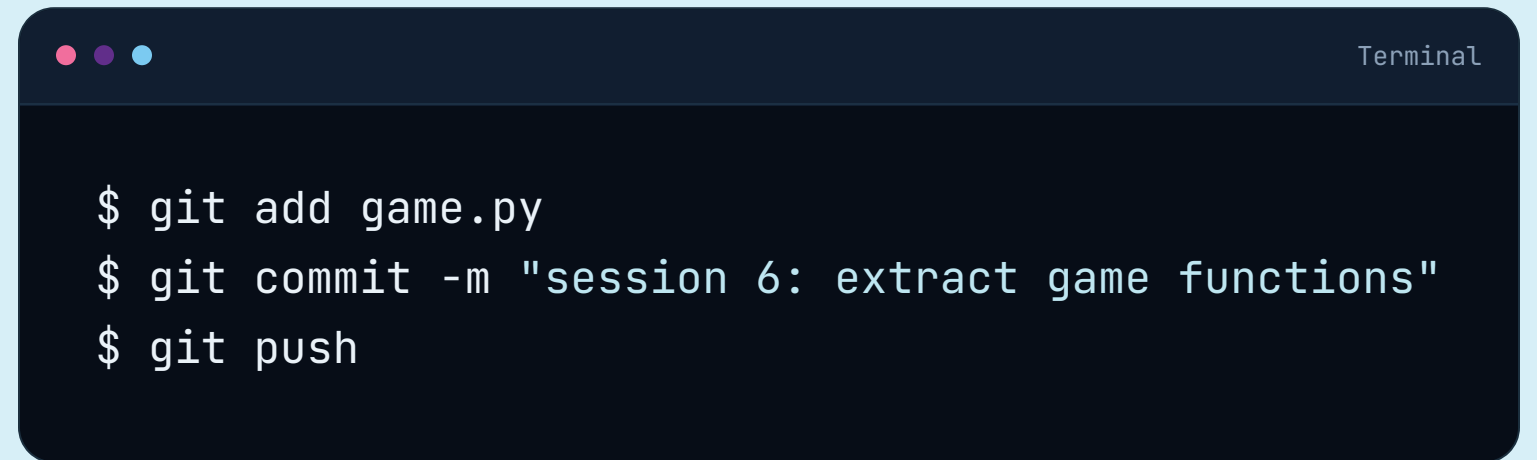
WRONG INDENTATION

```
def scan(destination):
```

Check: Is the body indented under def?

Commit the cleaner game loop.

Your history should explain that this was an organisation change.



```
Terminal  
  
$ git add game.py  
$ git commit -m "session 6: extract game functions"  
$ git push
```

CHECK GitHub shows the Session 6 commit.



// reflect explain the design

What changed if the game still plays the same?

01 Which function is pure, and why?

02 What does a parameter carry in?

03 What does return send back?



// optional after the core

Can one more job earn a name?

Extend the scanner with water data, or write a pure fuel_estimate() function.

NEXT: one Ship holds the scattered state